Chapter 4:

Pipelining and Parallelism

1. **How the performance of computer can be improved? Discuss the basic concepts of Pipelining.**

The speed of execution of programs is influenced by many factors. *One way to improve performance is to use faster circuit technology to build the processor and the main memory. Another possibility is to arrange the hardware so that more than one operation can be performed at the same time.* In this way, the number of operations performed per second is increased even though the elapsed time needed to perform any one operation is not changed.

Pipelining is a particularly effective w*ay of organizing concurrent activity in a computer system. It is a process of arrangement of hardware elements of the CPU such that its overall performance is increased. Simultaneous execution of more than one instruction takes place in a pipelined processor*.

Design of a basic pipeline

- *In a pipelined processor, a pipeline has two ends, the input end and the output end. Between these ends, there are multiple stages/segments such that output of one stage is connected to input of next stage and each stage performs a specific operation.*
- *Interface registers are used to hold the intermediate output between two stages. These interface registers are also called latch or buffer.*
- *All the stages in the pipeline along with the interface registers are controlled by a common clock.*

Consider how the idea of pipelining can be used in a computer. *The processor executes a program by fetching and executing instructions, one after the other*. Let F*i* and E*i* refer to the fetch and execute steps for instruction I*i*. Execution of a program consists of a sequence of fetch and execute steps, as shown in Figure 8.1*a*.

Lets consider a computer that has two separate hardware units, one for fetching instructions and another for executing them, as shown in Figure 8.1*b*. The instruction fetched by the fetch unit is deposited in an intermediate storage buffer, B1. This buffer is needed to enable the execution unit to execute the instruction while the

fetch unit is fetching the next instruction. The results of execution are deposited in the destination location specified by the instruction.

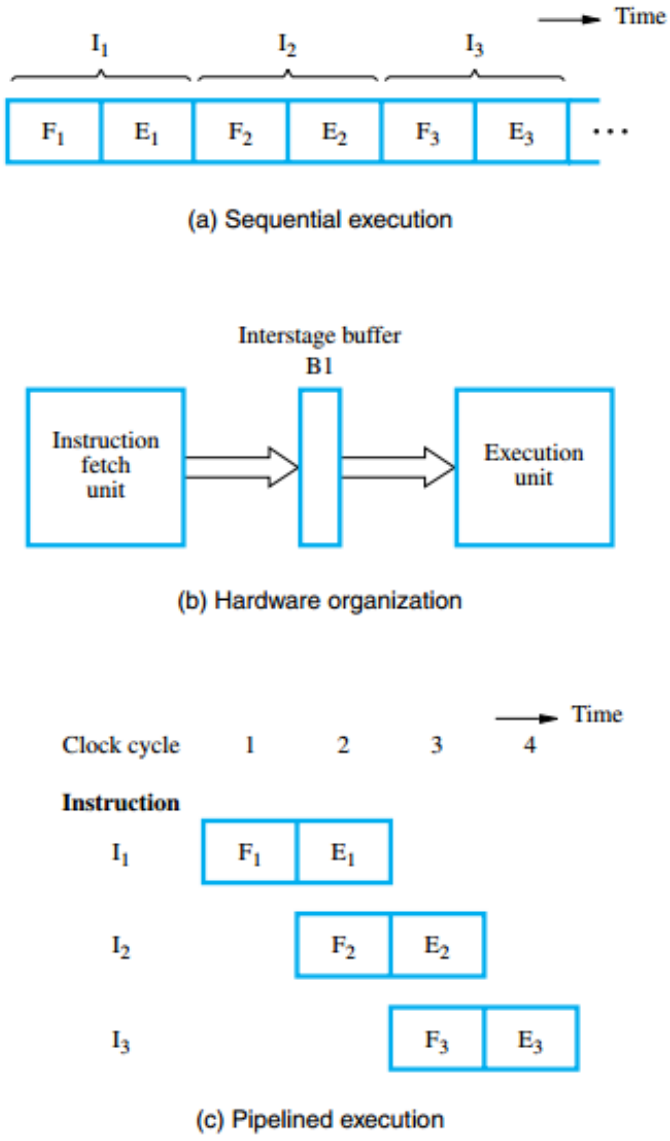


Figure 8.1 Basic idea of instruction pipelining.

The computer is controlled by a clock whose period is such that the fetch and execute steps of any instruction can each be completed in one clock cycle. Operation of the computer proceeds as in Figure 8.1*c*. In the first clock cycle, the fetch unit fetches an instruction I1 (step F1) and stores it in buffer B1 at the end of the clock cycle.

In the second clock cycle, the instruction fetch unit proceeds with the fetch operation for instruction I2 (step F2). Meanwhile, the execution unit performs the operation specified by instruction I1, which is available to it in buffer B1 (step E1). By the end of the second clock cycle, the execution of instruction I1 is completed and instruction I2 is available. Instruction I2 is stored in B1, replacing I1, which is no longer needed. Step E2 is performed by the execution unit during the third clock cycle, while instruction I3 is being fetched by the fetch unit.

In this manner, both the fetch and execute units are kept busy all the time. If the pattern in Figure 8.1*c* can be sustained for a long time, the completion rate of instruction execution will be twice that achievable by the sequential operation depicted in Figure 8.1*a*.

The processing of an instruction need not be divided into only two steps. For example, a pipelined processor may process each instruction in four steps, as follows:

F Fetch: read the instruction from the memory.

D Decode: decode the instruction and fetch the source operand(s).

E Execute: perform the operation specified by the instruction.

W Write: store the result in the destination location.

The sequence of events for this case is shown in Figure 8.2*a*. *Four instructions are in progress at any given time. This means that four distinct hardware units are needed, as shown in Figure 8.2*b. These units must be capable of performing their tasks simultaneously and without interfering with one another. Information is passed from one unit to the next through a storage buffer. As an instruction progresses through the pipeline, all the information needed by the stages downstream must be passed along.

For example, during clock cycle 4, the information in the buffers is as follows:

• Buffer B1 holds instruction I3, which was fetched in cycle 3 and is being decoded by the instruction-decoding unit.

• Buffer B2 holds both the source operands for instruction I2 and the specification of the operation to be performed. This is the information produced by the decoding hardware in cycle 3. The buffer also holds the information needed for the write step of instruction I2 (step W2). Even though it is not needed by stage E, this information must be passed on to stage W in the following clock cycle to enable that stage to perform the required Write operation.

• Buffer B3 holds the results produced by the execution unit and the destination information for instruction I1.

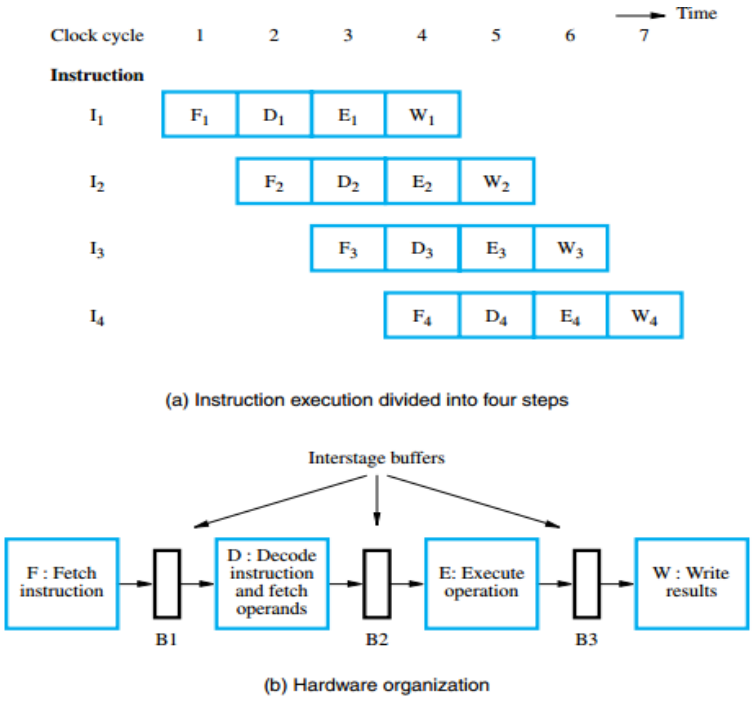


Figure 8.2 A 4-stage pipeline.

Execution in a pipelined processor

Execution sequence of instructions in a pipelined processor can be visualized using a space-time diagram. For example, consider a processor having 4 stages and let there

be 2 instructions to be executed. We can visualize the execution sequence through the following space-time diagrams:

Non overlapped execution:

Stage\Cycle	1	2	3	4	5	6	7	8
S1	I_1				I_2			
S2		I_1				I_2		
S3			I_1				I_2	
S4				I_1				I_2

Total time = 8 cycles

Overlapped execution:

Stage\Cycle	1	2	3	4	5
S1	I_1	I_2			
S2		I_1	I_2		
S3			I_1	I_2	
S4				I_1	I_2

Total time = 5 cycles

2. Derive the relation for speedup due to pipelining. What will be throughput.

Consider a 'k' segment pipeline with clock cycle time as 'Tp'. Let there be 'n' tasks to be completed in the pipelined processor. Now, the first instruction is going to take 'k' cycles to come out of the pipeline but the other 'n – 1' instructions will take only '1' cycle each, i.e, a total of 'n – 1' cycles. So, time taken to execute 'n' instructions in a pipelined processor:

$ET_{pipeline} = k + n - 1$ cycles

$= (k + n - 1)\ Tp$

In the same case, for a non-pipelined processor, execution time of 'n' instructions will be:

$ET_{non\text{-}pipeline} = n * k * Tp$

So, speedup (S) of the pipelined processor over non-pipelined processor, when 'n' tasks are executed on the same processor is:

S = Performance of pipelined processor / Performance of Non-pipelined processor

As the performance of a processor is inversely proportional to the execution time, we have,

$S = ET_{non\text{-}pipeline} / ET_{pipeline}$

$\Rightarrow S = [n * k * Tp] / [(k + n - 1) * Tp]$

$S = [n * k] / [k + n - 1]$

When the number of tasks 'n' are significantly larger than k, that is, $n >> k$

$S \approx n * k / n$

$S \approx k$

where 'k' are the number of stages in the pipeline.

Throughput = Number of instructions / Total time to complete the instructions
So, Throughput = $n / (k + n - 1) * Tp$

3. Discuss different types of pipeline.

Pipelines are usually divided into two classes: *instruction pipelines and arithmetic pipelines.* A pipeline in each of these classes can be designed in two ways: *static or dynamic.* A static pipeline can perform only one operation/function (such as addition or multiplication) at a time. The operation of a static pipeline can only be changed after the pipeline has been drained. (A pipeline is said to be drained when the last input data leave the pipeline.) it is composition of stages one after another means that the output of one stage is become input to the next stage we also called it linear pipelining. For example, consider a static pipeline that is able to perform addition and multiplication. Each time that the pipeline switches from a multiplication operation to an addition operation, it must be drained and set for the new operation. The performance of static pipelines is severely degraded when the operations change often, since this requires the pipeline to be drained and refilled each time. A dynamic pipeline can perform more than one operation at a time. To

perform a particular operation on an input data, the data must go through a certain sequence of stages. in it stages are connected in a liner fashion but this kind of pipelining used feedforward and feed backward connections as a input to the stages.it perform variable function but static perform fixed functions.in dynamic pipelining we can take intermediate outputs.

Arithmetic pipelines differ from instruction pipelines in some important ways. They are generally synchronous. This means that each stage executes in a fixed number of clock cycles. In a synchronous pipeline, moreover, no buffering between stages is provided. Each stage must be ready to accept the data passed from a previous stage when that data is produced. Another important difference is that an arithmetic pipeline may be nonlinear.

4. Discuss about DLX Pipeline.

DLX is a simple pipeline architecture for CPU. It is mostly used in universities as a model to study pipelining technique.

The architecture of DLX was chosen based on observations about most frequently used primitives in programs.

Every DLX instruction can be implemented in at most five clock cycles. The five clock cycles are

a. Instruction fetch cycle (IF)
b. Instruction decode/register fetch (ID)
c. Execution/Effective address cycle (EX)
d. Memory access/branch completion cycle (MEM)
e. Write-back cycle (WB)

DLX datapath with almost no changes by starting a new instruction on each clock cycle. Each of the clock cycles of the DLX datapath now becomes a pipe stage: a cycle in the pipeline.
While each instruction takes five clock cycles to complete, during each clock cycle the hardware will initiate a new instruction and will execute some part of the five different instructions. The typical way to show what is going on is:

Instr Num	1	2	3	4	5	6	7	8	9
instr i	IF	ID	EX	MEM	WB				
instr i+1		IF	ID	EX	MEM	WB			
instr i+2			IF	ID	EX	MEM	WB		
instr i+3				IF	ID	EX	MEM	WB	
instr i+4					IF	ID	EX	MEM	WB

5. Discuss various types of Pipeline Hazards and their solutions with examples.

There are mainly three types of dependencies possible in a pipelined processor. These are:
1) Structural Dependency
2) Control Dependency
3) Data Dependency

These dependencies may introduce stalls in the pipeline.

Stall: A stall is a cycle in the pipeline without new input.

Structural dependency
This dependency arises due to the resource conflict in the pipeline. A resource conflict is a situation when more than one instruction tries to access the same resource in the same cycle. A resource can be a register, memory, or ALU.

Example:

Instruction\ Cycle	1	2	3	4	5
I_1	IF(Mem)	ID	EX	Mem	
I_2		IF(Mem)	ID	EX	
I_3			IF(Mem)	ID	EX
I_4				IF(Mem)	ID

Resource conflict

In the above scenario, in cycle 4, instructions I_1 and I_4 are trying to access same resource (Memory) which introduces a resource conflict.
To avoid this problem, we have to keep the instruction on wait until the required resource (memory in our case) becomes available. This wait will introduce stalls in the pipeline as shown below:

Instruction\ Cycle	1	2	3	4	5	6	7	8
I_1	IF(Mem)	ID	EX	Mem	WB			
I_2		IF(Mem)	ID	EX	Mem	WB		
I_3			IF(Mem)	ID	EX	Mem	WB	
I_4				-	-	-	IF(Mem)	

3 Stalls

Solution for structural dependency
To minimize structural dependency stalls in the pipeline, we use a hardware mechanism called Renaming.

Renaming : According to renaming, we divide the memory into two independent modules used to store the instruction and data separately called Code memory(CM) and Data memory(DM) respectively. CM will contain all the instructions and DM will contain all the operands that are required for the instructions.

Data Hazards

Data hazards occur when two instructions in a pipeline refer to the same register and at least one of them writes to the register. Compiler writers use the phrase "data dependences" to cover the same kind of problem, but their terminology refers to what you can see in an instruction stream without considering the pipeline.

Also, execution circuitry is usually broken up into multiple functional units, each performing different types of operations. These functional units can be performing operations in parallel. More complex operations may take several cycles to complete. To illustrate the difficulties that result, consider the following MIPS code snippet.

```
div.d   $f0, $f2, $f4
mul.d   $f6, $f8, $f0
add.d   $f0, $f10, $f12
```

The use of register $f0 can give rise to three different kinds of problems in this code.

- Read after Write (RAW) hazards, also known as true dependences
- Write after Write (WAW) hazards, also known as output dependences
- Write after Read (WAR) hazards, also known as antidependences

The naming of these hazards is based on what is supposed to happen. That is, a WAR hazard occurs when an instruction that writes to a register follows soon after an instruction that reads from the same register. If the write precedes the read then the first instruction (the read) is working with the wrong data value.

Read after Write Hazards (True Dependences)

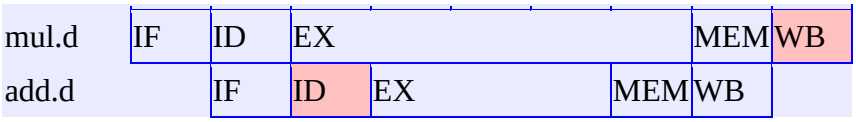
A *true dependence* arises when one instruction computes a value that is used by a later instruction. More precisely, the same operand register is a destination operand in the earlier instruction and a source operand in the later instruction, and there are no instructions between them that write a different value to the register.

If the two instructions can be in the pipeline at the same time and there is a possibility that the value will not be ready when the second instruction reads its source operands the condition is called a *read after write* (RAW) hazard.

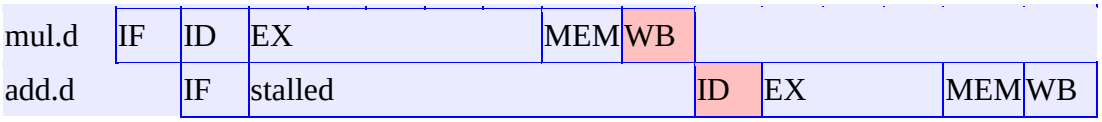
For example, consider the following code.

```
mul.d   $f0, $f2, $f4
add.d   $f6, $f8, $f0
```

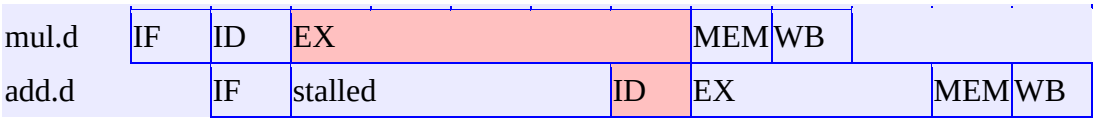
Here, the value produced by the mul.d instruction in $f0 is used as a source operand by the add.d instruction. Even if the execution phase of the mul.d instruction takes a single cycle, there is a hazard because the value is produced late but it is needed early as a source operand. Suppose the mul.d instruction takes 5 cycles for its EX stage and the add.d takes 3 cycles. The following chart indicates the timing of the two instructions with the RAW hazard ignored.



In order to handle the RAW hazard correctly, the ID stage of the add.d instruction should be stalled until $f0 has the value written by the mul.d instruction.



Register forwarding is a technique for faster handling of RAW hazards. It involves adding and controling direct data paths from functional unit outputs to functional unit inputs. With register forwarding, the ID stage of the add.d instruction can be started during the last cycle of the EX stage of the mul.d instruction, as shown below.



Write after Write Hazards (Output Dependences)

An *output dependence* arises when an earlier instruction writes a result to the same place that a later instruction writes to. More precisely, the same operand register is a destination operand in both the earlier instruction and the later instruction, and there are no instructions between them that write a different value to the register.

If the two instructions can be in the pipeline at the same time and there is a possibility that the second instruction will write its result before the first instruction writes its result then the condition is called a *write after write* (WAW) hazard.

For example, consider the following code.

```
mul.d   $f0, $f2, $f4
add.d   $f0, $f10, $f12
```

Here, the mul.d instruction and the add.d instructions will both try to write to $f0. Suppose the mul.d instruction takes 5 cycles for its EX stage and the add.d takes 3 cycles. The following chart indicates the timing of the two instructions.

mul.d	IF	ID	EX			MEM	WB
add.d		IF	ID	EX	MEM	WB	

This will result in later instructions seeing the wrong value in $f0 - the result from the mul.d instruction rather than the result of the add.d instruction. To remedy this problem, the WB stage of the add.d instruction should be stalled until after the WB stage of the mul.d instruction, as shown below.

mul.d	IF	ID	EX			MEM	WB
add.d		IF	ID	EX	MEM	stalled	WB

Write after Read Hazards (Antidependences)

An *antidependence* arises when an earlier instruction reads a value from the same place that will be written by a later instruction. That is, the same operand register is a source operand in the earlier instruction and a destination operand in the later instruction, and there are no instructions between them that write a different value to the register.

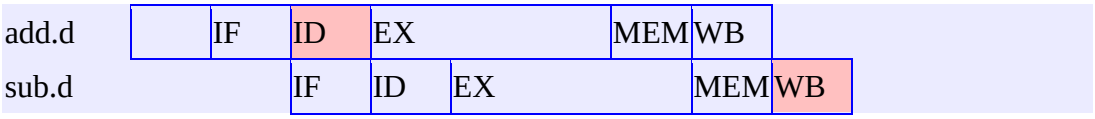
If the two instructions can be in the pipeline at the same time and there is a possibility that the later instruction will write its result before the earlier instruction has read its source operand value then the condition is called a *write after read* (WAR) hazard.

For example, consider the following code.

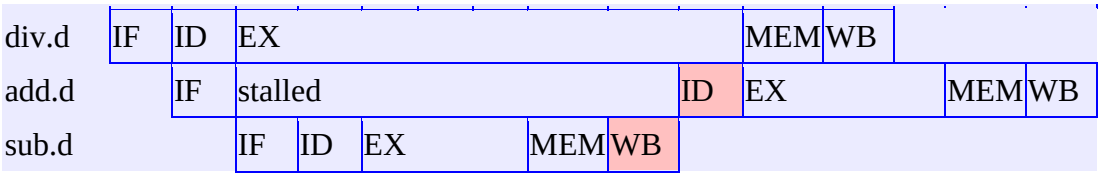
```
div.d   $f2, $f4, $f6
add.d   $f8, $f2, $f0
sub.d   $f0, $f10, $f12
```

Here, the value in $f0 is a source operand for the add.d instruction and a destination operand for the sub.d instruction. The add.d instruction executes incorrectly if it reads $f0 after the sub.d instruction has written to $f0. The following chart seems to indicate that this cannot happen. It assumes that the EX stage for a div.d instruction takes 8 cycles and the EX stage for a add.d or a sub.d instruction takes 3 cycles.

div.d	IF	ID	EX					MEM	WB



However, the WAR hazard arises when the register read for the add.d instruction is delayed due to its RAW hazard with the div.d instruction regarding $f2. The following chart shows the real situation, assuming that register forwarding is used.



Now the write from the sub.d instruction changes the value in $f0 before the add.d has had a chance to read the original value. The problem arises here because the add.d has a source operand, $f2, that must be read late due to a RAW hazard, while another source operand, $f0, should be read early to avoid a WAR hazard. If the two reads can be done at different times then the WAR hazard disappears.

Control Hazard.
In any set of instructions, there is normally a need for some kind of statement that allows the flow of control to be something other than sequential. Instructions that do this are included in every programming language and are called *branches*. In general, about 30% of all instructions in a program are branches. This means that branch instructions in the pipeline can reduce the throughput tremendously if not handled properly.

Whenever a branch is taken, *the performance of the pipeline is seriously affected. Each such branch requires a new address to be loaded into the program counter, which may invalidate all the instructions that are either already in the pipeline or prefetched i*n the buffer. This draining and refilling of the pipeline for each branch degrade the throughput of the pipeline to that of a sequential processor.

It is to be noted *that the presence of a branch statement does not automatically cause the pipeline to drain and begin refilling. A branch not taken allows the continued sequential flow of uninterrupted instructions to the pipeline*. Only when a branch is taken does the problem arise.

In general, branch instructions can be classified into three groups: *(1) unconditional branch, (2) conditional branch.*
An unconditional branch always alters the sequential program flow. It sets a new target address in the program counter, rather than incrementing it by 1 to point to the next sequential instruction address, as is normally the case.
A conditional branch sets a new target address in the program counter only when a certain condition, usually based on a condition code, is satisfied. Otherwise, the program counter is incremented by 1 as usual. In other words, a conditional branch selects a path of instructions based on a certain condition. If the condition is

satisfied, the path starts from the target address and is called a *target path*. If it is not, the path starts from the next sequential instruction and is called a *sequential path*.

Conditional branches are the hardest to handle. As an example, consider the following conditional branch instruction sequence:

*i*1
*i*2 (conditional branch to *i*k)
*i*3
.
.
.
*i*k (target)
*i*k+1

Figure 3.11 shows the execution of this sequence in our instruction pipeline when the target path is selected. In this figure, *c* denotes the branch penalty, that is, the number of cycles wasted whenever the
target path is chosen.

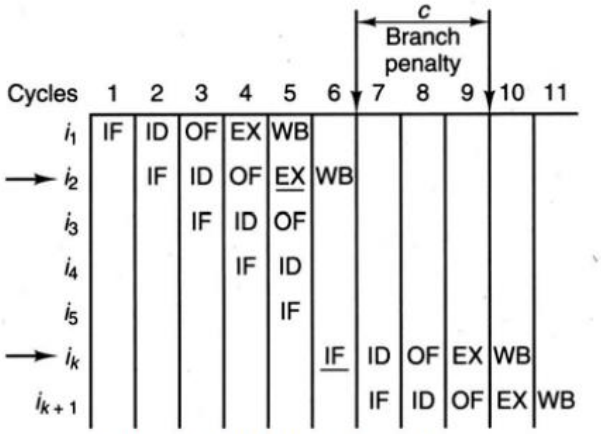


Figure 3.11 Branch Penalty.

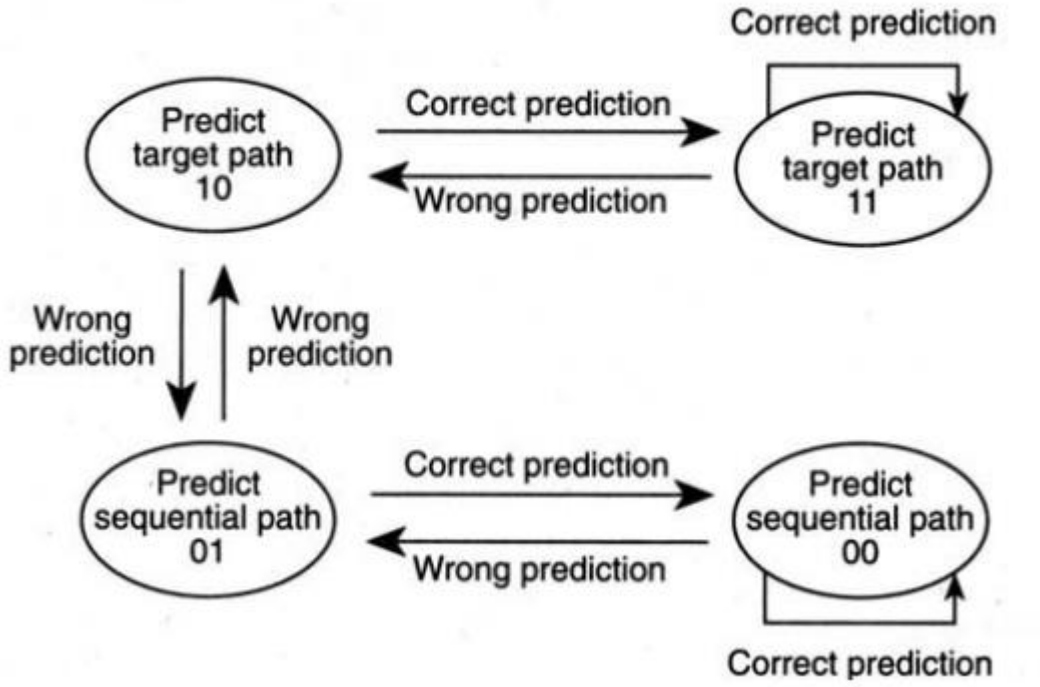
Some of the better known techniques are branch prediction, delayed branching, and multiple prefetching. Each of these techniques is explained next.

Branch Prediction.
In this type of design, the outcome of a *branch decision is predicted before the branch is actually executed*. Therefore, based on a particular prediction, the sequential path or the target path is chosen for execution. Although the chosen path often reduces the branch penalty, it may increase the penalty in case of incorrect prediction.
There are two types of predictions, static and dynamic.
In static prediction, a fixed decision for *prefetching one of the two paths* is made before the program runs. For example, a simple technique would be to always assume that the branch is taken. This technique simply loads the program counter with the target address

when a branch is encountered. Another such technique is to automatically choose one path (sequential or target) for some branch types and another for the rest of the branch types. If the chosen path is wrong, the pipeline is drained and instructions corresponding to the correct path are fetched; the penalty is paid.

In dynamic prediction, during the execution of the program the processor makes a *decision based on the past information* of the previously executed branches. For example, a simple technique would be to record the history of the last two paths taken by each branch instruction. If the last two executions of a branch instruction have chosen the same path, that path will be chosen for the current execution of the branch instruction. If the two paths do not match, one of the paths will be chosen randomly.

A better approach is to associate an n-bit counter with each branch instruction. This is known as the counter-based branch prediction approach. In this method, after executing a branch instruction for the first time, its counter, C, is set to a threshold, T, if the target path was taken, or to T-1 if the sequential path was taken. From then on, whenever the branch instruction is about to be executed, if $C \geq T$, then the target path is taken; otherwise, the sequential path is taken. The counter value C is updated after the branch is resolved. If the correct path is the target path, the counter is incremented by 1; if
not, C is decremented by 1. If C ever reaches $2n$-1 (an upper bound), C is no longer incremented, even if the target path was correctly predicted and chosen. Likewise, C is never decremented to a value less than 0.
In practice, often n and T are chosen to be 2. Studies have shown that 2-bit predictors perform almost as well as predictors with more number of bits. The following diagram represents the possible states in a 2-bit predictor.



An alternative scheme to the preceding 2-bit predictor is to change the prediction only when the predicted path has been wrong for two consecutive times.

Most processors employ a small size cache memory called *branch target buffer* (BTB); sometimes referred to as *target instruction cache* (TIC). Often, each entry of this cache keeps a branch instruction's address with its target address and the history used by the prediction scheme.

When a branch instruction is first executed, the processor allocates an entry in the BTB for this instruction. When a branch instruction is
fetched, the processor searches the BTB to determine whether it holds an entry for the corresponding branch instruction. If there is a hit, the recorded history is used to determine whether the sequential or target path should be taken.

Static prediction methods usually require little hardware, but they may increase the complexity of the compiler. In contrast, dynamic prediction methods increase the hardware complexity, but they require less work at compile time.

In general, dynamic prediction obtains better results than static prediction and also provides a greater degree of object code compatibility, since decisions are made after compile time.

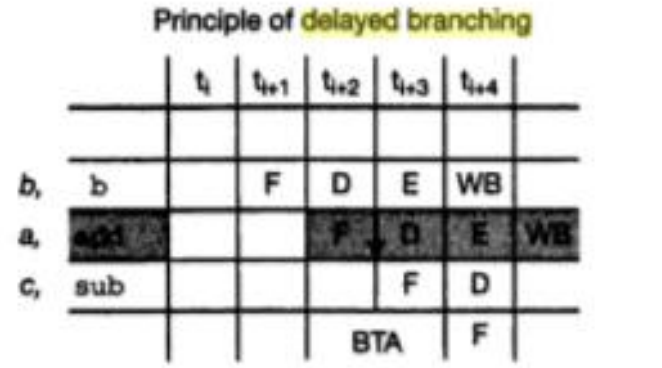
Delayed Branching.

The delayed branching scheme eliminates or significantly reduces the effect of the branch penalty. In this type of design, a certain number of instructions after the

branch instruction is fetched and executed regardless of which path will be chosen for the branch.
For example, a processor with a branch delay of *k* executes a path containing the next *k* sequential instructions and then either continues on the same path or starts a new path from a new target address.

As often as possible, the compiler tries to fill the next *k* instruction slots after the branch with instructions that are independent from the branch instruction. NOP (no operation) instructions are placed in any remaining empty slots.

As an example, consider the following case:



Branching to the target instruction (sub) is executed with one pipeline cycle of delay. This cycle is utilized to execute the instruction in the delay slot (add). Thus, delayed branching results in the following execution sequence:

```
a,  add
b,    b
c,  sub
```

This scheme is applicable to both conditional and unconditional branches.

Multiple Prefetching.

In this type of design, the processor fetches both possible paths. Once the branch decision is made, the unwanted path is thrown away.

By prefetching both possible paths, the fetch penalty is avoided in the case of an incorrect prediction. *To fetch both paths, two buffers are employed to service the pipeline.* In normal execution, the first buffer is loaded with instructions from the next sequential address of the branch instruction. *If a branch occurs, the contents of the first buffer are invalidated, and the secondary buffer, which has been loaded with instructions from the target address of the branch instruction, is used as the primary buffer.*

This double buffering scheme ensures a constant flow of instructions and data to the pipeline and reduces the time delays caused by the draining and refilling of the pipeline.

List the challenges of implementing pipelining?

Challenges of implementing pipelining can be listed as follows:

a. *Because the divide unit is not fully pipelined, structural hazards can occur*
b. *Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1*
c. *WAW hazards are possible, since instructions no longer reach WB in order WAR hazards are NOT possible, since register reads are still taking place during the ID stage*
d. *Instructions can complete in a different order than they were issued, causing problems with exceptions*
e. *Longer latency of operations makes stalls for RAW hazards more frequent*

6. What is impact of pipelining in Instruction Set Design?

RISC machines like MIPS are well suited for pipelining. Instruction format is simple and execution relatively uniform. Pipelining is more complex for CISC machines, because the instructions may take different lengths of time to execute. However, RISC-style pipelining is now incorporated into high-performance CISC processors (such as the Pentium and Core 2) by translating most instructions into a series of RISC-like operations.

Execution of an instruction in a pipelined processor differs from execution in a non pipelined processor. Most of the instructions execute smoothly in pipeline without reducing the pipeline performance. But, some instructions either reduce pipeline performance or cause side effects. Since all modern processors are pipelined, impact of various instructioins in pipeline execution should be taken into consideration while finalizing the instruction set of processors.

Addressing modes.

Complex addressing modes are avoided in pipelined processors. Instructions using complex addressing modes take long execution time and likely to cause pipeline stall. A processor designer prefers to have following features in addressing modes:

1. Operand fetch should not require more than one memory access

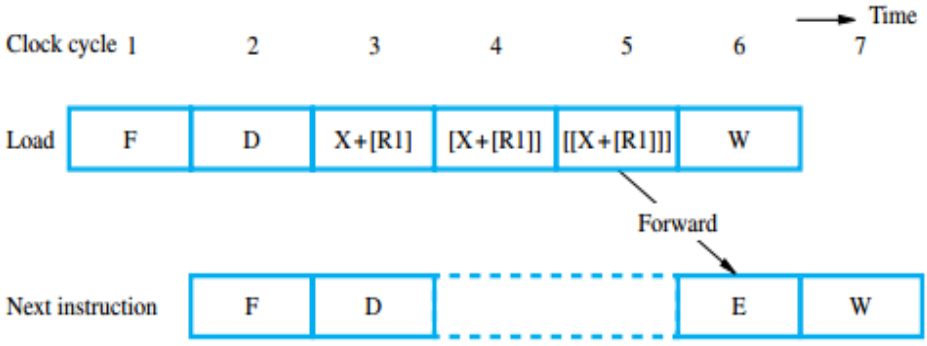
2. No side effects should be caused

3. Instructions other than load and store should not access memory for operand fetch or result store.

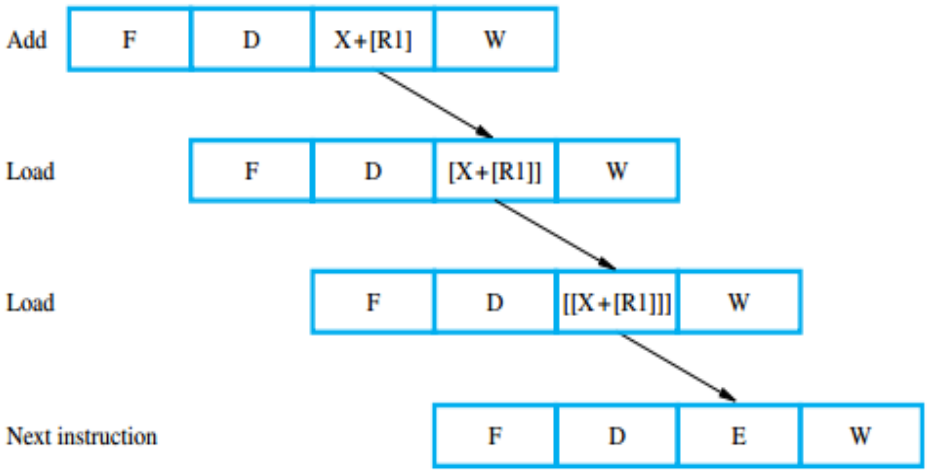
These feature are present in three basic addressing modes: register addressing, register indirect addressing and index addressing. They do not cause side effects. The index addressing mode involves operand address calculation in one clock cycle, and the operand fetch in next clock cycle. The other two addressing modes do not involve operand address calculation.

Condition codes:

While compiler tries to reorder the instructions to avoid pipeline stall, it should not change the program logic due to instruction reordering. The condition codes set (or reset) by certain instruction should be taken into account by the compiler so as not to disturb the program logic due to instruction reordering. To simplify the compiler's task, the instruction set designer should choose only few instructions that affect condition codes. In addition, the compiler should have provision to indicate the instruction which have affected the condition codes.



(a) Complex addressing mode



(b) Simple addressing mode

Figure 8.16 Equivalent operations using complex and simple addressing modes.

7. **Explain concepts and challenges of Instruction Level Parallelism**.

   Instruction-level parallelism (ILP) is the potential overlap the execution of instructions using pipeline concept to improve performance of the system. The various techniques that are used to increase amount of parallelism are reduces the impact of data and control hazards and increases processor ability to exploit parallelism

   There are two approaches to exploiting ILP.

   1. Static Technique – Software Dependent
   2. Dynamic Technique – Hardware Dependent

The simplest and most common way to increase the amount of parallelism is loop-level parallelism. Here is a simple example of a loop, which adds two 1000- element arrays, that is completely parallel:

for (i=1;i<=1000; i=i+1) x[i] = x[i] + y[i];

CPI (Cycles per Instruction) for a pipelined processor is the sum of the base CPI and all contributions from stalls:

Pipeline CPI = Ideal pipeline CPI + Structural stalls + Data hazard stalls + Control stalls

The ideal pipeline CPI is a measure of the maximum performance attainable by the implementation. By reducing each of the terms of the right-hand side, we minimize the overall pipeline CPI and thus increase the IPC (Instructions per Clock).

Technique	Reduces
Forwarding and bypassing	Potential data hazard stalls
Delayed branches and simple branch scheduling	Control hazard stalls
Basic dynamic scheduling (scoreboarding)	Data hazard stalls from true dependences
Dynamic scheduling with renaming	Data hazard stalls and stalls from anti dependences and output dependences
Dynamic branch prediction	Control stalls
Issuing multiple instructions per cycle	Ideal CPI
Speculation	Data hazard and control hazard stalls
Dynamic memory disambiguation	Data hazard stalls with memory
Loop unrolling	Control hazard stalls
Basic compiler pipeline scheduling	Data hazard stalls
Compiler dependence analysis	Ideal CPI, data hazard stalls
Compiler speculation	Ideal CPI, data, control stalls

Various types of Dependences in ILP.

Data Dependence and Hazards:

To exploit instruction-level parallelism, determine which instructions can be executed in parallel. If two instructions are parallel, they can execute simultaneously in a pipeline without causing any stalls. If two instructions are dependent they are not parallel and must be executed in order.

There are three different types of dependences: data dependences (also called true data dependences), name dependences, and control dependences.

Data Dependences:

An instruction j is data dependent on instruction i if either of the following holds:

• Instruction i produces a result that may be used by instruction j, or

• Instruction j is data dependent on instruction k, and instruction k is data dependent on instruction i.

The second condition simply states that one instruction is dependent on another if there exists a chain of dependences of the first type between the two instructions. This dependence chain can be as long as the entire program.

For example, consider the following code sequence that increment a vector of values in memory (starting at 0(R1) and with the last element at 8(R2)) by a scalar in register F2:

```
Loop: L.D    F0,0(R1) ;    F0=array element
ADD.D        F4,F0,F2 ;    add scalar in F2
S.D     F4,0(R1) ;    store result
DADDUI     R1,R1,#-8 ; decrement pointer 8 bytes
BNE  R1,R2,LOOP ; branch R1!=zero
```

The dependence implies that there would be a chain of one or more data hazards between the two instructions. Executing the instructions simultaneously will cause a processor with pipeline interlocks to detect a hazard and stall, thereby reducing or eliminating the overlap. Dependences are a property of programs.

Whether a given dependence results in an actual hazard being detected and whether that hazard actually causes a stall are

properties of the pipeline organization. This difference is critical to understanding how instruction-level parallelism can be exploited.

The presence of the dependence indicates the potential for a hazard, but the actual hazard and the length of any stall is a property of the pipeline. The importance of the data dependences is that a dependence

(1) indicates the possibility of a hazard,

(2) Determines the order in which results must be calculated, and

(3) Sets an upper bound on how much parallelism can possibly be exploited.

Name Dependences

The name dependence occurs when two instructions use the same register or memory location, called a name, but there is no flow of data between the instructions associated with that name.

There are two types of name dependences between an instruction i that precede instruction j in program order:

- An antidependence between instruction i and instruction j occurs when instruction j writes a register or memory location that instruction i reads. The original ordering must be preserved to ensure that i reads the correct value.

- An output dependence occurs when instruction i and instruction j write the same register or memory location. The ordering between the instructions must be preserved to ensure that the value finally written corresponds to instruction j.

Both anti-dependences and output dependences are name dependences, as opposed to true data dependences, since there is no value being transmitted between the instructions. Since a name dependence is not a true dependence, instructions involved in a name dependence can execute simultaneously or be reordered, if the name (register number or memory location) used in the instructions is changed so the instructions do not conflict.

This renaming can be more easily done for register operands, where it is called register renaming. Register renaming can be done either statically by a compiler or dynamically by the hardware. Before describing dependences arising from branches, let’s examine the relationship between dependences and pipeline data hazards.

Control Dependences:

A control dependence determines the ordering of an instruction, i, with respect to a branch instruction so that the instruction i is executed in correct program order. Every

Instruction, except for those in the first basic block of the program, is control dependent on some set of branches, and, in general, these control dependences must be preserved to preserve program order. One of the simplest examples of a control dependence is the dependence of the statements in the “then” part of an if statement on the branch. For example, in the code segment:

if p1 { S1; };

if p2 { S2;}

S1 is control dependent on p1, and S2is control dependent on p2 but not on p1. In general, there are two constraints imposed by control dependences:

1. An instruction that is control dependent on a branch cannot be moved before the branch so that its execution is no longer controlled by the branch. For example, we cannot take an instruction from the then-portion of an if-statement and move it before the if statement.

2. An instruction that is not control dependent on a branch cannot be moved after the branch so that its execution is controlled by the branch. For example, we cannot take a statement before the if-statement and move it into the then-portion.

Control dependence is preserved by two properties in a simple pipeline, First, instructions execute in program order. This ordering ensures that an instruction that occurs before a branch is executed before the branch. Second, the detection of control or branch hazards ensures that an instruction that is control dependent on a branch is not executed until the branch direction is known.

2. Data Hazard and various hazards in ILP.

Data Hazards

A hazard is created whenever there is dependence between instructions, and they are close enough that the overlap caused by pipelining, or other reordering of instructions, would change the order of access to the operand involved in the dependence.

Because of the dependence, preserve order that the instructions would execute in, if executed sequentially one at a time as determined by the original source program. The goal of both our software and hardware techniques is to exploit parallelism by preserving program order only where it affects the outcome of the program. Detecting and avoiding hazards ensures that necessary program order is preserved.

Data hazards may be classified as one of three types, depending on the order of read and write accesses in the instructions.

Consider two instructions i and j, with i occurring before j in program order. The possible data hazards are

RAW (read after write) — j tries to read a source before i write it, so j incorrectly gets the old value. This hazard is the most common type and corresponds to true data dependence. Program order must be preserved to ensure that j receives the value from i. In the simple common five-stage static pipeline a load instruction followed by an integer ALU instruction that directly uses the load result will lead to a RAW hazard.

WAW (write after write)

j tries to write an operand before it is written by i. The writes end up being performed in the wrong order, leaving the value written by i rather than the value written by j in the destination. This hazard corresponds to output dependence. WAW hazards are present only in pipelines that write in more than one pipe stage or allow an instruction to proceed even when a previous instruction is stalled. The classic five-stage integer pipeline writes a register only in the WB stage and avoids this class of hazards.

WAR (write after read)

j tries to write a destination before it is read by i, so i incorrectly gets the new value. This hazard arises from antidependence. WAR hazards cannot occur in most static issue pipelines even deeper pipelines or floating point pipelines because all reads are early (in ID) and all writes are late (in WB). A WAR hazard occurs either

when there are some instructions that write results early in the instruction pipeline, and other instructions that read a source late in the pipeline or when instructions are reordered.

8. How data hazards can be overcame using dynamic scheduling?

Data hazards in a program cause a processor to stall. With static scheduling the compiler tries to reorder these instructions during compile time to reduce pipeline stalls. It uses less hardware and can use more powerful algorithms. With dynamic scheduling the hardware tries to rearrange the instructions during run-time to reduce pipeline stalls. It uses simpler compiler and handles dependencies not known at compile time. It allows code compiled for a different machine to run efficiently.

Dynamic scheduling offers several advantages.

1. First, it allows code that was compiled with one pipeline to run efficiently on a different pipeline, eliminating the need to recompile for a different microarchitecture.
2. Second, it enables handling of code when dependences are unknown at time of compilation.
3. Third, and most important, it allows the processor to tolerate unpredictable delays.

Instruction issue: The process of letting an instruction move from the instruction decode stage (ID) into the execution stage (EX)of this pipeline.

- An instruction that has made this step is said to have *issued*.
- A pipeline fetches an instruction and issues it,
  - if there is no *data dependence* between an instruction already in the pipeline and the fetched instruction that cannot be hidden with forwarding.
  - If there is a data dependence that cannot be hidden, then the hazard detection hardware stalls the pipeline (starting with the instruction that uses the result).

•*Forwarding* logic reduces the effective pipeline latency so that the certain dependences do not result in hazards. *No* new instructions are fetched or issued until the dependencies cleared.

In case of Static scheduling: Compiler techniques can be used for scheduling the instructions to separate dependent instructions.
While in dynamic scheduling: the hardware rearranges the instruction execution to reduce the stalls.
Its advantages can be listed as follows:

- It simplifies the compiler
- It enables handling some cases when dependences are unknown at compile time.
- It allows code that was compiled with one pipeline in mind to run efficiently on a different pipeline.

The disadvantage is increase in hardware complexity.

Dynamically scheduled processor tries to avoid stalling when dependences are present.

In our previous model, all instructions executed in the order that they appear This can lead to unnecessary stalls. Consider following sets of instructions:

DIVD FO, F2, F4

ADDD F10, F0, F8

SUBD F12, F8, F14

Here, SUBD stalls waiting for the ADDD to go first, even though SUBD does not have a data dependency.

Scoreboarding

The scoreboard implements a centralized control scheme that detects all resource and data hazards which allows instructions to execute out-of-order when no resource hazards or data dependencies. With out-of-order execution, the SUBD is allowed to executed before the add. This can lead to out of order completion, which can cause WAW and WAR hazards.

The solution for WAW is to detect WAW hazard before reading operands and stall write until other instruction completes.

Similarly, the Solutions for WAR is to detect WAR hazards before writing back to the register files and stall the write back

It was first implemented in 1964 by the CDC 6600. This scoreboard does not take advantage of forwarding (i.e. bypasses), since it waits until both results are written back to the register file. It's pipelining has four stages as follows:

- Decode+Issue (Issue)
  - decode instructions
  - check for structural and WAW hazards
  - stall until structural and WAW hazards are resolved
- Read operands (Read)
  - wait until no RAW hazards
  - then read operands
- Execution (EX)
  - operate on operands
  - may be multiple cycles
  - notify scoreboard when done
- Write result (WB)

– finish execution

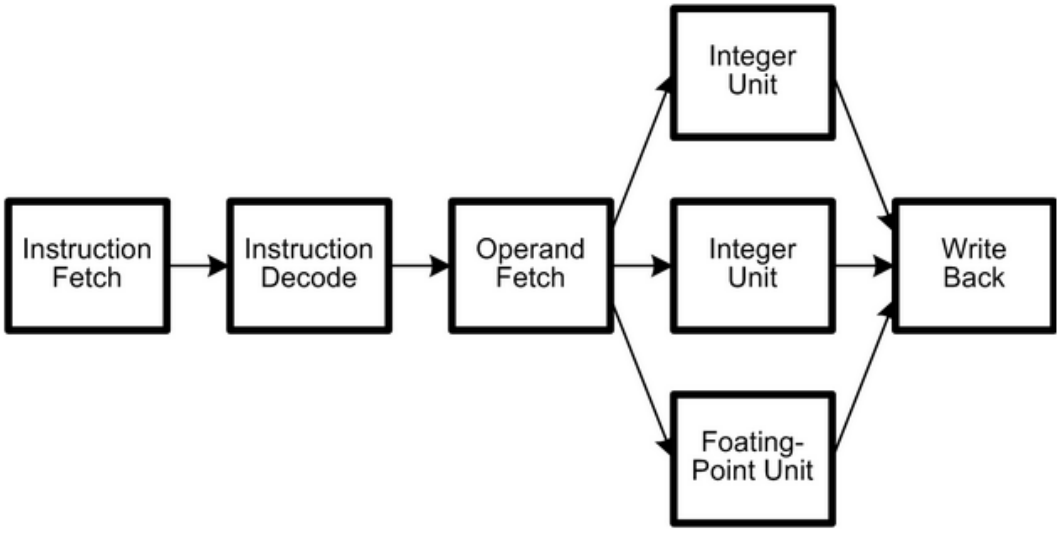
– stall if WAR hazard

9. Discuss superscalar and super pipelining architecture.

Superscalar Architecture (SSA) describes a microprocessor design that execute more than one instruction at a time during a single clock cycle. In a SSA design, the processor or the instruction compiler is able to determine whether an instruction can be carried out independently of other sequential instructions, or whether it has a dependency on another instruction and must be executed sequentially. The design is sometimes called “Second Generation RISC”. Another term used to describe superscalar processors is multiple instruction issue processors.

In a SSA, several scalar instructions can be initiated simultaneously and executed independently. A long series of innovations aimed at producing ever-faster microprocessors. It includes all features of pipelining but, in addition, there can be several instructions executing simultaneously in the same pipeline stage.

SSA introduces a new level of parallelism, called instruction-level parallelism. In Superscalar CPU Architecture implementation of Instruction Level Parallelism (ILP) within a single processor allows faster CPU at a given clock rate. A superscalar processor executes more than one instruction during a clock cycle by simultaneously dispatching multiple instructions to functional units. Each functional unit is not a separate CPU core but an execution resource within a single CPU such as an arithmetic logic unit, a bit shifter, or a multiplier.



A Superscalar Processor with 3 Functional Units

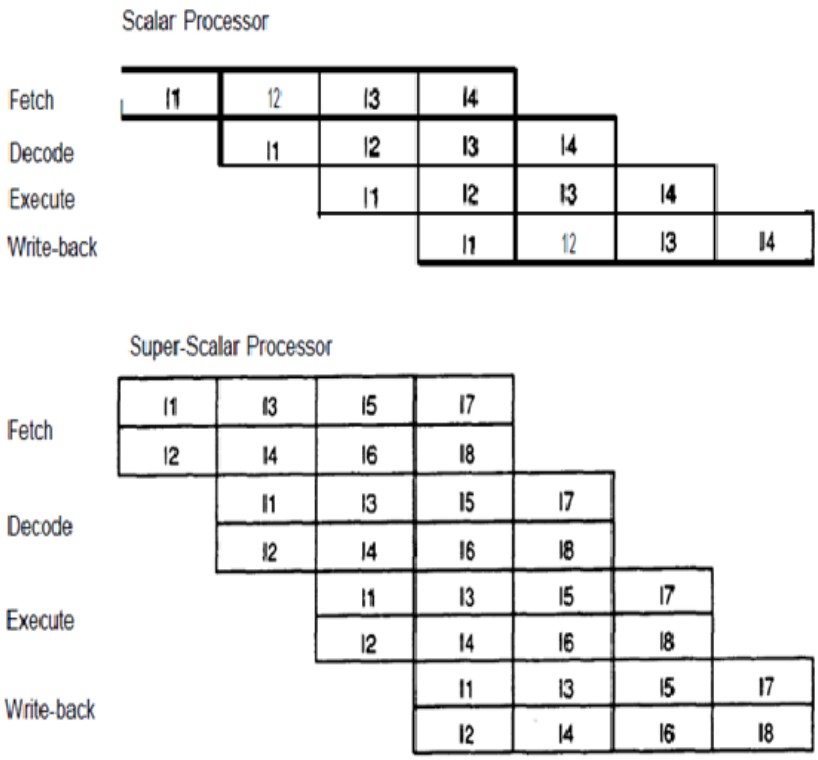
The simplest processors are scalar processors. Each instruction executed by a scalar processor typically manipulates one or two data items at a time. In a superscalar CPU the dispatcher reads instructions from memory and decides which one can be run in

parallel. Therefore, a superscalar processor can be proposed having multiple parallel pipelines, each of which is processing instructions simultaneously from a single instruction thread.

Pipelining in Superscalar Architecture

Pipelining is the process of breaking down task into sub steps and executing them in different parts of processor. In order to fully utilise a superscalar processor of degree *m* with pipelining, *m* instructions must be executable parallely. This situation may not be true in all clock cycles.

In a superscalar processor, the simple operation latency should require only one cycle, as in the base scalar processor. Pipelining is the process of breaking down task into sub steps and executing them in different parts of processor. Following diagram shows the pipelined execution in scalar and super scalar processer:



Effect of Dependencies

The situations which prevent instructions to be executed in parallel by SSA are very similar to those which prevent efficient execution on a pipelined architecture (pipeline hazards):

- Resource conflicts.
- Control (procedural) dependency.
- Data dependencies.

Their consequences on SSA are more severe than those on simple pipelines, because the potential of parallelism in SSA is greater and, thus, a larger amount of performance will be lost.

Resource conflicts.

Several instructions compete for the same hardware resource at the same time.

e.g., two arithmetic instructions need the same floating-point unit for execution.

It is similar to structural hazards in pipeline. They can be solved partly by introducing several hardware units for the same functions. e.g., have two floating-point units. The hardware units can also be pipelined to support several operations at the same time.

Procedural conflicts

The presence of branches creates major problems in assuring the optimal parallelism. It cannot execute instructions after a branch in parallel with instructions before a branch. It is similar to control hazards in pipeline. If instructions are of variable length, they cannot be fetched and issued in parallel, since an instruction has to be decoded in order to identify the following one. Therefore, superscalar techniques are more efficiently applicable to RISCs, with fixed instruction length and format.

Data conflicts

It is caused by data dependencies between instructions in the program.

It is similar to date hazards in pipeline. To address the problem and to increase the degree of parallel execution, SSA provides a great liberty in the order in which instructions are issued and executed. Therefore, data dependencies have to be considered and dealt with much more carefully.

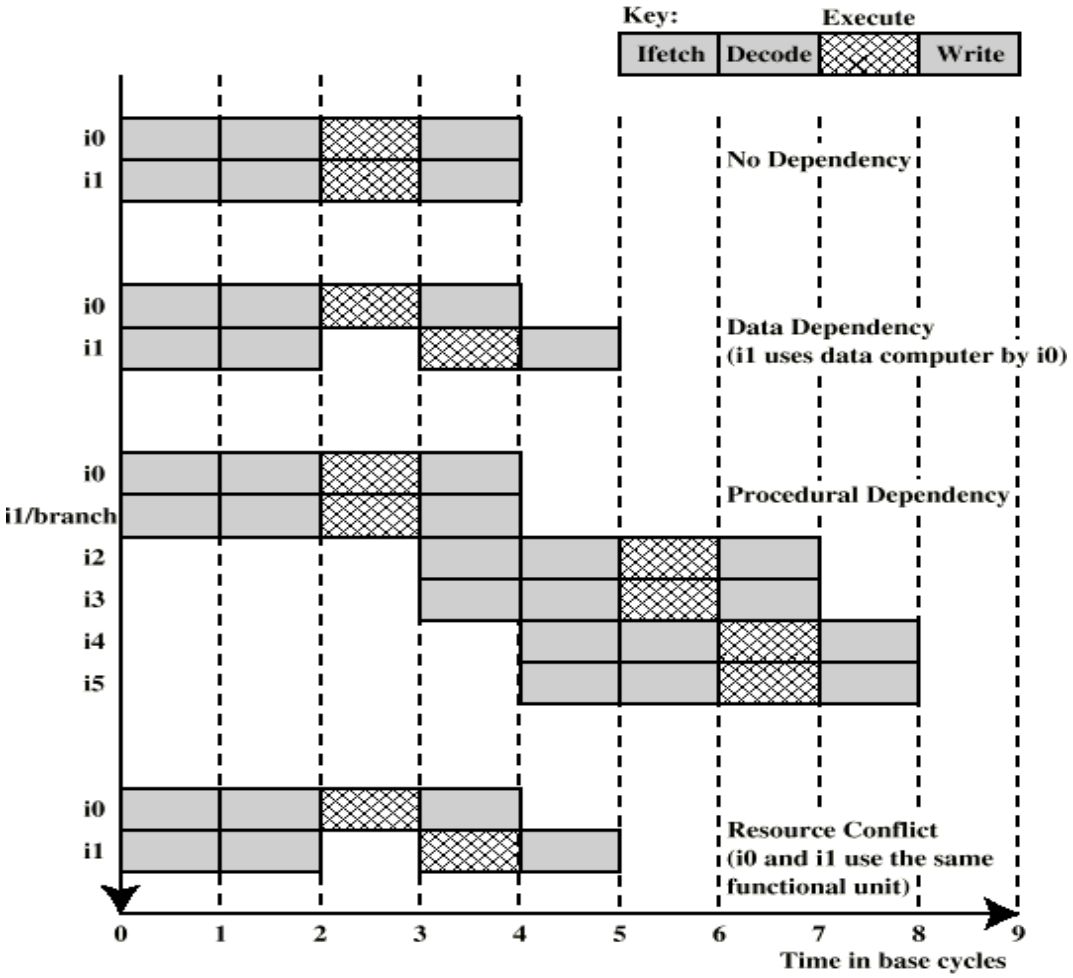
Due to data dependencies, only some part of the instructions are potential subjects for parallel execution. In order to find instructions to be issued in parallel, the processor has to select from a sufficiently large instruction sequence.

There are usually a lot of data dependencies in a short instruction sequence.

Window of execution is defined as the set of instructions that is considered for execution at a certain moment. The number of instructions in the window should be as large as possible. However, this is limited by:

- Capacity to fetch instructions at a high rate.
- The problem of branches.
- The cost of hardware needed to analyze data dependencies.

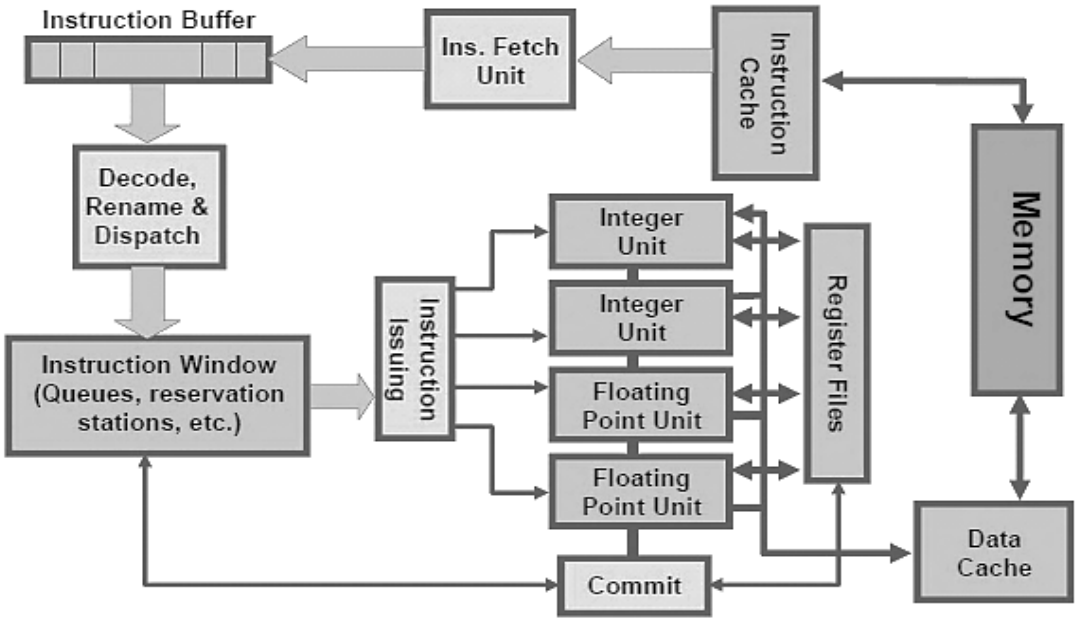
Following diagram shows the effects of dependency in superscalar architecture:



Instruction Flow in Superscalar Architecture

SSA allows several instructions to be issued and completed per clock cycle. It consists of a number of pipelines that are working in parallel. Depending on the number and kind of parallel units available, a certain number of instructions can be

executed in parallel. In the following example two floating point and two integer operations can be issued and executed simultaneously. Each unit is also pipelined and can execute several operations in different pipeline stages.



Superpipelining

Superpipelining is based on dividing the stages into several sub-stages, and thus increasing the number of instructions which are handled by the pipeline at the same time.

For example, by dividing each stage into two sub-stages, a pipeline can perform at twice the speed in the ideal situation:

- Tasks that require less than half a clock cycle.
- No duplication of hardware is needed for these stages.

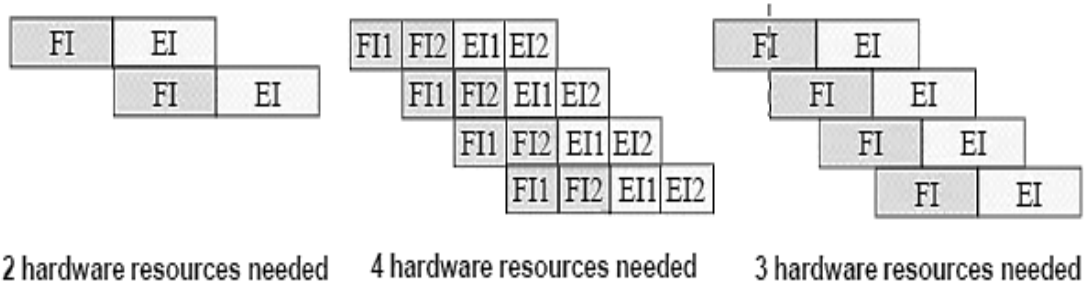


Figure: Duplication of hardware is for Superscalar

Benefit:

1. The major benefit of superpipelining is the increase in the number of instructions which can be in the pipeline at one time and hence the level of parallelism.

Drawbacks

1. The larger number of instructions "in flight" (*ie* in some part of the pipeline) at any time, increases the potential for data dependencies to introduce stalls. Simulation studies have suggested that a pipeline depth of more than 8 stages tends to be counter-productive.
2. Superscalar machines can issue several instructions per cycle. Superpipelined machines can issue only one instruction per cycle, but they have cycle times shorter than the time required for any operation.
3. Both of these techniques exploit instruction-level parallelism, which is often limited in many applications. Superpipelined machines are shown to have better performance and less cost than superscalar machines.

Superscalar vs. Superpipeline

Base (scalar) machine: 4-stage pipeline

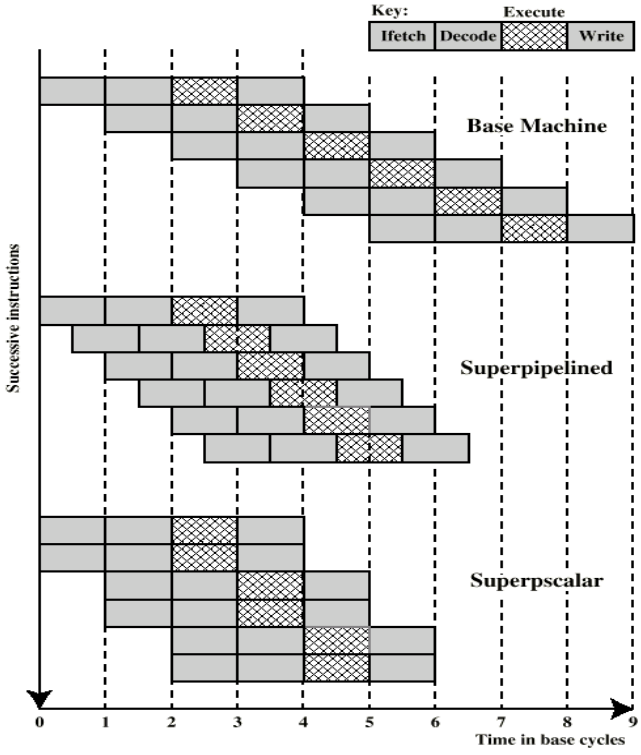
- Instruction fetch
- Operation decode
- Operation execution
- Result write back

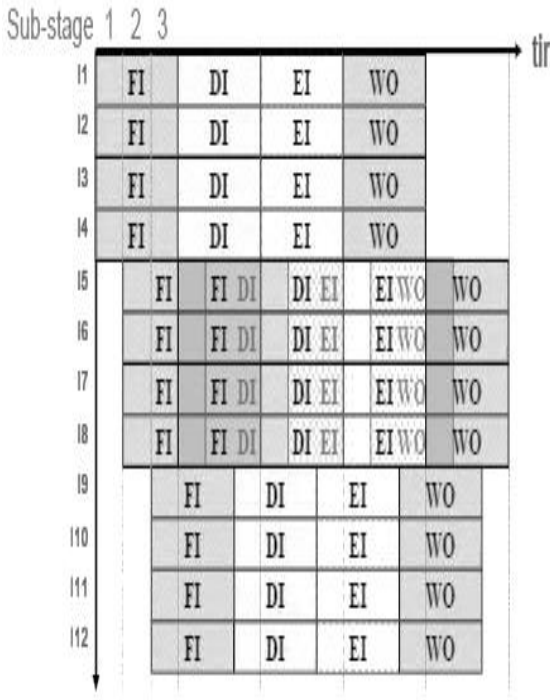
Superpipeline of degree 2

- A sub-stage often takes half a clock cycle to finish.

Superscalar of degree 2

- Two instructions are executed.
- Duplication of hardware is required by definition.





Superpipeline of degree 3 is 12 times speed-up over the base machine while and superscalar of degree 4 is 48 times speedup over sequential execution

10. Why is RISC architecture better suited for pipeline processing than CISC?

RISC uses simple instructions which can be executed within one clock cycle, whereas primary goal of CISC is to complete a task in as few lines as possible.

For example, consider task of multiplying two numbers. CISC architecture would come prepared with a specific instruction for doing this task. Whereas in RISC you would need to perform three or four instructions for completing the same task.

RISC architectures lend themselves more towards pipelining than CISC architectures for many reasons. As RISC architectures have a smaller set of instructions than CISC architectures in a pipeline architecture the time required to fetch and decode for CISC architectures is unpredictable.

The difference in instruction length with CISC will hinder the fetch decode sections of a pipeline, a single byte instruction following an 8 byte instruction will need to be handled so as not to slow down the whole pipeline. In RISC architectures the fetch and decode cycle is more predictable and most instructions have similar length.

CISC architectures by their very name also have more complex instructions with complex addressing modes. This makes the whole cycle of processing an instruction more complex. Pipelining requires that the whole fetch to execute cycle can be split into stages where each stage does not interfere with the next and each instruction can be doing something at each stage. RISC architectures because of their simplicity and small set of instructions are simple to split into stages. CISC with more complex instructions are harder to split into stages. Stages that are important for one instruction may be not be required for another instruction with CISC.

The rich set of addressing modes that are available in CISC architectures can cause data hazards when pipelining is introduced. Data hazards which are unlikely to occur in RISC architectures due to the smaller subset of instructions and the use of load store instruction to access memory become a problem in CISC architectures. CISC instructions that write results back to memory need to be handled carefully. Forwarding solutions used for allowing result written to registers to be available for input in the next instruction become more complex when memory locations which can be addressed in various modes can be accessed. Write after Read hazard must be taken care of where CISC instruction may auto increment a register early in the stages which may be used by the previous instruction at a later stage.

CISC added complexity makes for larger pipeline lengths to take into account more decoding and checking. This is because with a large number of logic gates for a fetch execute cycle the additional gates required for stages have less impact. As RISC architectures have simpler instruction sets than CISC the number of gates involved in the fetch execute cycle compare will be far lower than this in CISC architecture. Therefore, RISC architectures will tend to have smaller optimum pipeline lengths than more general processors.

RISC architectures do suite pipelining more than CISC architectures and do lend themselves to smaller pipelines. This does not mean however that CISC architecture cannot gain from pipelining or that a large number of pipeline stages are bad (although the flushing of a pipeline would become of concern).

OLD QUESTIONS

1. Explain the use of pipelining. Differentiate between RISC and CISC pipelining with suitable diagrams.
2. Draw the block diagram of General Superscalar Processor and explain it. Differentiate between superscalar and Super pipeline with required block diagram.
3. Differentiate between RISC and CISC architecture with suitable examples.
4. Explain different types of dependencies and conflicts in the instruction level parallelism with suitable examples.
5. Explain about normal and delayed branch in the RISC machine with suitable codes and diagrams. Differentiate between registers and cache.
6. What do you mean by procedural dependencies and resource conflicts? Explain dependencies in a superscalar processor with suitable diagram. What are the process for superscalar implementation?
7. Why pipelining is required in a computer? **Differentiate between RISC and CISC pipelining with suitable diagrams**.
8. Differentiate between Superscalar and Superpipeline with required diagram. Draw the block diagram of General Superscalar Processor and explain it.
9. What do you understanding by pipelining? What is a RISC Pipelining? Explain the effect of RISC pipelining with suitable example.

10. What is a superscalar processor? Draw the block diagram of general superscalar processor and explain it. Differentiate between superscalar and superpipeline with required block diagram
11. What do you mean by RISC pipelining? Explain with suitable diagram.